

Introduction to Graph Theory in Java DSA

Graph theory is a fundamental area of computer science and mathematics that deals with the study of graphs, which are mathematical structures used to model pairwise relations between objects. In Java, graphs can be implemented using various data structures, and they are essential for solving problems related to networks, paths, and connectivity.

Key Concepts

- **Vertex (Node):** A fundamental part of a graph, representing an entity (e.g., a city in a map).
- **Edge:** A connection between two vertices, representing a relationship (e.g., a road between two cities).
- **Weighted Graph:** A graph where edges have weights (e.g., distances).
- **Directed Graph:** A graph where edges have a direction (e.g., one-way streets).
- **Undirected Graph:** A graph where edges do not have a direction (e.g., two-way streets).

Graph Representation

Graphs can be represented in several ways in Java:

1. **Adjacency Matrix:** A 2D array where the cell at row *i* and column *j* indicates the presence of an edge between vertex *i* and vertex *j*.
2. **Adjacency List:** An array of lists where each list contains the neighbors of a vertex.

Example Implementation

Below is a simple implementation of a graph using an adjacency list in Java, along with Depth First Search (DFS) and Breadth First Search (BFS) algorithms.

Graph Class

```
java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2
3class Graph {
4    private int vertices; // Number of vertices
5    private LinkedList<Integer> adjList[]; // Adjacency list
6
7    // Constructor
8    Graph(int v) {
9        vertices = v;
10        adjList = new LinkedList[v];
11        for (int i = 0; i < v; i++) {
12            adjList[i] = new LinkedList<>();
13        }
14    }
15
16    // Add edge to the graph
17    void addEdge(int v, int w) {
18        adjList[v].add(w); // Add w to v's list
19    }
20
21    // Depth First Search
22    void DFS(int v) {
23        boolean visited[] = new boolean[vertices];
```

```

24     DFSUtil(v, visited);
25 }
26
27 // Utility function for DFS
28 private void DFSUtil(int v, boolean visited[]) {
29     visited[v] = true;
30     System.out.print(v + " ");
31
32     for (Integer neighbor : adjList[v]) {
33         if (!visited[neighbor]) {
34             DFSUtil(neighbor, visited);
35         }
36     }
37 }
38
39 // Breadth First Search
40 void BFS(int start) {
41     boolean visited[] = new boolean[vertices];
42     LinkedList<Integer> queue = new LinkedList<>();
43
44     visited[start] = true;
45     queue.add(start);
46
47     while (queue.size() != 0) {
48         int v = queue.poll();
49         System.out.print(v + " ");
50
51         for (Integer neighbor : adjList[v]) {
52             if (!visited[neighbor]) {
53                 visited[neighbor] = true;
54                 queue.add(neighbor);
55             }
56         }
57     }
58 }
59}

```

Driver Code

```

java
VerifyOpen In EditorRunCopy code
1public class GraphDemo {
2    public static void main(String[] args) {
3        Graph graph = new Graph(7); // Create a graph with 7 vertices
4
5        // Add edges
6        graph.addEdge(0, 1);
7        graph.addEdge(0, 4);
8        graph.addEdge(1, 2);
9        graph.addEdge(1, 3);
10       graph.addEdge(1, 4);
11       graph.addEdge(2, 3);
12       graph.addEdge(3, 4);
13       graph.addEdge(4, 5);
14       graph.addEdge(5, 6);
15
16       System.out.println("Depth First Traversal (starting from vertex 0):");
17       graph.DFS(0); // Perform DFS starting from vertex 0
18
19       System.out.println("\nBreadth First Traversal (starting from vertex
20       0):");
21       graph.BFS(0); // Perform BFS starting from vertex 0
22   }
23 }

```

Explanation of the Code

- **Graph Class:** This class contains methods to add edges and perform DFS and BFS traversals.
- **DFS Method:** Uses recursion to visit nodes. It marks nodes as visited and prints them.
- **BFS Method:** Uses a queue to explore nodes level by level, marking them as visited.

Conclusion

Graph theory is a powerful tool in computer science, and understanding how to implement and traverse graphs is essential for solving complex problems. The above example provides a foundational understanding of graph representation and traversal algorithms in Java. You can expand upon this by implementing additional features such as weighted edges, directed graphs, and more complex algorithms like Dijkstra's or A* for pathfinding.

Certainly! Let's explore **graph terminology** and the **types of graphs** in the context of Java Data Structures and Algorithms (DSA), along with examples.

Graph Terminology

- **Graph:** A graph is a non-linear data structure consisting of a set of vertices (or nodes) and edges that connect pairs of vertices. It can be represented as $G = (V, E)$, where (V) is the set of vertices and (E) is the set of edges.
- **Vertex (Node):** A fundamental unit of a graph representing an entity. For example, in a social network, each user can be a vertex.
- **Edge:** A connection between two vertices. Edges can be directed (one-way) or undirected (two-way).
- **Degree of a Vertex:** The number of edges incident to a vertex. In directed graphs, it is further classified into in-degree (incoming edges) and out-degree (outgoing edges).
- **Path:** A sequence of vertices where each adjacent pair is connected by an edge.
- **Cycle:** A path that starts and ends at the same vertex without repeating any vertices (except the starting/ending vertex).

Types of Graphs

1. **Undirected Graph:** Edges have no direction. Example: A friendship graph where connections are mutual.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1// Undirected Graph Representation
2class Graph {
3    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
```

```

4
5     public void addEdge(int v1, int v2) {
6         adjList.putIfAbsent(v1, new ArrayList<>());
7         adjList.putIfAbsent(v2, new ArrayList<>());
8         adjList.get(v1).add(v2);
9         adjList.get(v2).add(v1); // Undirected
10    }
11}

```

2. Directed Graph (Digraph): Edges have a direction. Example: A web page link structure.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1// Directed Graph Representation
2class DirectedGraph {
3    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
4
5    public void addEdge(int from, int to) {
6        adjList.putIfAbsent(from, new ArrayList<>());
7        adjList.get(from).add(to); // Directed
8    }
9}

```

3. Weighted Graph: Edges have weights (costs). Example: A road network where weights represent distances.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class WeightedGraph {
2    private final Map<Integer, List<Edge>> adjList = new
HashMap<>();
3
4    static class Edge {
5        int to;
6        int weight;
7
8        Edge(int to, int weight) {
9            this.to = to;
10           this.weight = weight;
11        }
12    }
13
14    public void addEdge(int from, int to, int weight) {
15        adjList.putIfAbsent(from, new ArrayList<>());
16        adjList.get(from).add(new Edge(to, weight));
17    }
18}

```

4. Unweighted Graph: Edges do not have weights. Example: A simple social network graph.

- **Example:** Similar to the undirected graph example above.

5. Complete Graph: Every pair of vertices is connected by an edge. Example: A tournament graph.

- **Example:**

```

java

```

```

VerifyOpen In EditorRunCopy code
1class CompleteGraph {
2    private final int vertices;
3
4    public CompleteGraph(int vertices) {
5        this.vertices = vertices;
6    }
7
8    public void printEdges() {
9        for (int i = 0; i < vertices; i++) {
10            for (int j = i + 1; j < vertices; j++) {
11                System.out.println("Edge: " + i + " - " + j);
12            }
13        }
14    }
15}

```

6. **Bipartite Graph:** Vertices can be divided into two disjoint sets such that every edge connects a vertex in one set to a vertex in the other.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class BipartiteGraph {
2    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
3
4    public void addEdge(int u, int v) {
5        adjList.putIfAbsent(u, new ArrayList<>());
6        adjList.putIfAbsent(v, new ArrayList<>());
7        adjList.get(u).add(v);
8        adjList.get(v).add(u);
9    }
10}

```

7. **Directed Acyclic Graph (DAG):** A directed graph with no cycles. Example: Task scheduling.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class DAG {
2    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
3
4    public void addEdge(int from, int to) {
5        adjList.putIfAbsent(from, new ArrayList<>());
6        adjList.get(from).add(to); // Directed
7    }
8}

```

Conclusion

Graphs are versatile data structures used in various applications, from social networks to transportation systems. Understanding the terminology and types of graphs is crucial for implementing algorithms and solving problems effectively in Java DSA.

If you have any specific questions or need further examples, feel free to ask!

Certainly! Let's explore how to represent a graph using both an **adjacency matrix** and an **adjacency list** in Java, along with examples for each representation.

1. Adjacency Matrix Representation

An adjacency matrix is a 2D array where the rows and columns represent the vertices of the graph. If there is an edge between vertex *i* and vertex *j*, the matrix entry `matrix[i][j]` is set to 1 (or `true`), otherwise it is 0 (or `false`).

Java Code Example for Adjacency Matrix

```
java
VerifyOpen In EditorRunCopy code
1public class Graph {
2    private boolean[][] adjMatrix;
3    private int numVertices;
4
5    // Constructor to initialize the graph with a given number of vertices
6    public Graph(int numVertices) {
7        this.numVertices = numVertices;
8        adjMatrix = new boolean[numVertices][numVertices];
9    }
10
11    // Method to add an edge between two vertices
12    public void addEdge(int i, int j) {
13        adjMatrix[i][j] = true; // For directed graph
14        adjMatrix[j][i] = true; // Uncomment for undirected graph
15    }
16
17    // Method to remove an edge between two vertices
18    public void removeEdge(int i, int j) {
19        adjMatrix[i][j] = false;
20        adjMatrix[j][i] = false; // Uncomment for undirected graph
21    }
22
23    // Method to print the adjacency matrix
24    public void printGraph() {
25        for (int i = 0; i < numVertices; i++) {
26            for (int j = 0; j < numVertices; j++) {
27                System.out.print((adjMatrix[i][j] ? 1 : 0) + " ");
28            }
29            System.out.println();
30        }
31    }
32
33    public static void main(String[] args) {
34        Graph graph = new Graph(4);
35        graph.addEdge(0, 1);
36        graph.addEdge(1, 2);
37        graph.addEdge(2, 0);
38        graph.addEdge(2, 3);
39
40        System.out.println("Adjacency Matrix:");
41        graph.printGraph();
42    }
43}
```

2. Adjacency List Representation

An adjacency list is a collection of lists or arrays where each list corresponds to a vertex and contains a list of its adjacent vertices. This representation is more space-efficient for sparse graphs.

Java Code Example for Adjacency List

```
java
VerifyOpen In EditorRunCopy code
1import java.util.ArrayList;
2
3public class Graph {
4    private ArrayList<ArrayList<Integer>> adjList;
5    private int numVertices;
6
7    // Constructor to initialize the graph with a given number of vertices
8    public Graph(int numVertices) {
9        this.numVertices = numVertices;
10       adjList = new ArrayList<>(numVertices);
11       for (int i = 0; i < numVertices; i++) {
12           adjList.add(new ArrayList<>());
13       }
14   }
15
16   // Method to add an edge between two vertices
17   public void addEdge(int i, int j) {
18       adjList.get(i).add(j); // For directed graph
19       adjList.get(j).add(i); // Uncomment for undirected graph
20   }
21
22   // Method to print the adjacency list
23   public void printGraph() {
24       for (int i = 0; i < numVertices; i++) {
25           System.out.print(i + ": ");
26           for (int j : adjList.get(i)) {
27               System.out.print(j + " ");
28           }
29           System.out.println();
30       }
31   }
32
33   public static void main(String[] args) {
34       Graph graph = new Graph(4);
35       graph.addEdge(0, 1);
36       graph.addEdge(1, 2);
37       graph.addEdge(2, 0);
38       graph.addEdge(2, 3);
39
40       System.out.println("Adjacency List:");
41       graph.printGraph();
42   }
43}
```

Summary

- **Adjacency Matrix:** Suitable for dense graphs, allows $O(1)$ time complexity for edge existence checks, but uses $O(V^2)$ space.
- **Adjacency List:** More space-efficient for sparse graphs, allows easy iteration over neighbors, but edge existence checks take $O(V)$ time in the worst case.

Feel free to run the provided code examples to see how each representation works! If you have any further questions or need additional examples, let me know!

Certainly! Let's explore **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** algorithms in Java, along with examples for each.

1. Breadth-First Search (BFS)

BFS Overview:

- BFS is a traversal algorithm that explores all the neighbor nodes at the present depth prior to moving on to nodes at the next depth level.
- It uses a **queue** data structure to keep track of nodes to visit next.

BFS Algorithm Steps:

1. Start from the root node (or any arbitrary node in the case of a graph).
2. Enqueue the starting node and mark it as visited.
3. While the queue is not empty:
 - Dequeue a node from the queue.
 - Visit the node (process it).
 - Enqueue all its unvisited neighbors and mark them as visited.

Java Implementation of BFS:

```
java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2import java.util.Queue;
3
4class Graph {
5    private int V; // Number of vertices
6    private LinkedList<Integer>[] adj; // Adjacency list
7
8    Graph(int v) {
9        V = v;
10        adj = new LinkedList[v];
11        for (int i = 0; i < v; i++) {
12            adj[i] = new LinkedList<>();
13        }
14    }
15
16    void addEdge(int v, int w) {
17        adj[v].add(w); // Add w to v's list
18    }
19
20    void BFS(int start) {
21        boolean[] visited = new boolean[V]; // Track visited nodes
22        Queue<Integer> queue = new LinkedList<>(); // Queue for BFS
23
24        visited[start] = true; // Mark the starting node as visited
25        queue.add(start); // Enqueue the starting node
26
27        while (!queue.isEmpty()) {
28            int node = queue.poll(); // Dequeue a node
29            System.out.print(node + " "); // Process the node
30
31            // Enqueue all unvisited neighbors
```



```

32         for (int neighbor : adj[node]) {
33             if (!visited[neighbor]) {
34                 visited[neighbor] = true; // Mark as visited
35                 queue.add(neighbor); // Enqueue
36             }
37         }
38     }
39 }
40
41 public static void main(String[] args) {
42     Graph graph = new Graph(6);
43     graph.addEdge(0, 1);
44     graph.addEdge(0, 3);
45     graph.addEdge(0, 4);
46     graph.addEdge(4, 5);
47     graph.addEdge(3, 5);
48     graph.addEdge(1, 2);
49
50     System.out.println("Breadth First Traversal starting from vertex 0:");
51     graph.BFS(0);
52 }
53}

```

Output:

```

VerifyOpen In EditorRunCopy code
1Breadth First Traversal starting from vertex 0:
20 1 3 4 2 5

```

2. Depth-First Search (DFS)

DFS Overview:

- DFS is a traversal algorithm that explores as far as possible along each branch before backtracking.
- It can be implemented using recursion or a stack.

DFS Algorithm Steps:

1. Start from the root node (or any arbitrary node).
2. Mark the node as visited.
3. For each unvisited neighbor, recursively perform DFS.

Java Implementation of DFS (using recursion):

```

java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2
3class GraphDFS {
4    private int V; // Number of vertices
5    private LinkedList<Integer>[] adj; // Adjacency list
6
7    GraphDFS(int v) {
8        V = v;
9        adj = new LinkedList[V];
10        for (int i = 0; i < v; i++) {
11            adj[i] = new LinkedList<>();
12        }
13    }
14

```

```

15 void addEdge(int v, int w) {
16     adj[v].add(w); // Add w to v's list
17 }
18
19 void DFS(int node, boolean[] visited) {
20     visited[node] = true; // Mark the node as visited
21     System.out.print(node + " "); // Process the node
22
23     // Recur for all the vertices adjacent to this vertex
24     for (int neighbor : adj[node]) {
25         if (!visited[neighbor]) {
26             DFS(neighbor, visited); // Recursive call
27         }
28     }
29 }
30
31 public static void main(String[] args) {
32     GraphDFS graph = new GraphDFS(6);
33     graph.addEdge(0, 1);
34     graph.addEdge(0, 3);
35     graph.addEdge(0, 4);
36     graph.addEdge(4, 5);
37     graph.addEdge(3, 5);
38     graph.addEdge(1, 2);
39
40     boolean[] visited = new boolean[6]; // Track visited nodes
41     System.out.println("Depth First Traversal starting from vertex 0:");
42     graph.DFS(0, visited);
43 }
44}

```

Output:

[Verify](#)
[Open In Editor](#)
[Run](#)
[Copy code](#)
 1Depth First Traversal starting from vertex 0:
 20 1 2 3 5 4

Summary

- **BFS** is best for finding the shortest path in unweighted graphs and exploring nodes level by level.
- **DFS** is useful for exploring all possible paths and is often used in scenarios like maze solving, topological sorting, and cycle detection.

Feel free to ask if you have any questions or need further examples!

Final Answer

Introduction to Graph Theory in Java DSA

Graph theory is a fundamental area of computer science and mathematics that deals with the study of graphs, which are mathematical structures used to model pairwise relations between objects. In Java, graphs can be implemented using various data structures, and they are essential for solving problems related to networks, paths, and connectivity.

Key Concepts

- **Vertex (Node):** A fundamental part of a graph, representing an entity (e.g., a city in a map).
- **Edge:** A connection between two vertices, representing a relationship (e.g., a road between two cities).
- **Weighted Graph:** A graph where edges have weights (e.g., distances).
- **Directed Graph:** A graph where edges have a direction (e.g., one-way streets).
- **Undirected Graph:** A graph where edges do not have a direction (e.g., two-way streets).

Graph Representation

Graphs can be represented in several ways in Java:

1. **Adjacency Matrix:** A 2D array where the cell at row *i* and column *j* indicates the presence of an edge between vertex *i* and vertex *j*.
2. **Adjacency List:** An array of lists where each list contains the neighbors of a vertex.

Example Implementation

Below is a simple implementation of a graph using an adjacency list in Java, along with Depth First Search (DFS) and Breadth First Search (BFS) algorithms.

Graph Class

```
java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2
3class Graph {
4    private int vertices; // Number of vertices
5    private LinkedList<Integer> adjList[]; // Adjacency list
6
7    // Constructor
8    Graph(int v) {
9        vertices = v;
10       adjList = new LinkedList[v];
11       for (int i = 0; i < v; i++) {
12           adjList[i] = new LinkedList<>();
13       }
14    }
15
16    // Add edge to the graph
17    void addEdge(int v, int w) {
18        adjList[v].add(w); // Add w to v's list
19    }
20
21    // Depth First Search
22    void DFS(int v) {
23        boolean visited[] = new boolean[vertices];
24        DFSUtil(v, visited);
25    }
26
27    // Utility function for DFS
28    private void DFSUtil(int v, boolean visited[]) {
29        visited[v] = true;
30        System.out.print(v + " ");
31
32        for (Integer neighbor : adjList[v]) {
33            if (!visited[neighbor]) {
```

```

34         DFSUtil(neighbor, visited);
35     }
36 }
37 }
38
39 // Breadth First Search
40 void BFS(int start) {
41     boolean visited[] = new boolean[vertices];
42     LinkedList<Integer> queue = new LinkedList<>();
43
44     visited[start] = true;
45     queue.add(start);
46
47     while (queue.size() != 0) {
48         int v = queue.poll();
49         System.out.print(v + " ");
50
51         for (Integer neighbor : adjList[v]) {
52             if (!visited[neighbor]) {
53                 visited[neighbor] = true;
54                 queue.add(neighbor);
55             }
56         }
57     }
58 }
59}

```

Driver Code

```

java
VerifyOpen In EditorRunCopy code
1public class GraphDemo {
2    public static void main(String[] args) {
3        Graph graph = new Graph(7); // Create a graph with 7 vertices
4
5        // Add edges
6        graph.addEdge(0, 1);
7        graph.addEdge(0, 4);
8        graph.addEdge(1, 2);
9        graph.addEdge(1, 3);
10       graph.addEdge(1, 4);
11       graph.addEdge(2, 3);
12       graph.addEdge(3, 4);
13       graph.addEdge(4, 5);
14       graph.addEdge(5, 6);
15
16       System.out.println("Depth First Traversal (starting from vertex 0):");
17       graph.DFS(0); // Perform DFS starting from vertex 0
18
19       System.out.println("\nBreadth First Traversal (starting from vertex
0):");
20       graph.BFS(0); // Perform BFS starting from vertex 0
21   }
22}

```

Explanation of the Code

- **Graph Class:** This class contains methods to add edges and perform DFS and BFS traversals.
- **DFS Method:** Uses recursion to visit nodes. It marks nodes as visited and prints them.
- **BFS Method:** Uses a queue to explore nodes level by level, marking them as visited.

Conclusion

Graph theory is a powerful tool in computer science, and understanding how to implement and traverse graphs is essential for solving complex problems. The above example provides a foundational understanding of graph representation and traversal algorithms in Java. You can expand upon this by implementing additional features such as weighted edges, directed graphs, and more complex algorithms like Dijkstra's or A* for pathfinding.

Graph Terminology and Types of Graphs in Java DSA

Graph Terminology

- **Graph:** A graph is a non-linear data structure consisting of a set of vertices (or nodes) and edges that connect pairs of vertices. It can be represented as $G = (V, E)$, where (V) is the set of vertices and (E) is the set of edges.
- **Vertex (Node):** A fundamental unit of a graph representing an entity. For example, in a social network, each user can be a vertex.
- **Edge:** A connection between two vertices. Edges can be directed (one-way) or undirected (two-way).
- **Degree of a Vertex:** The number of edges incident to a vertex. In directed graphs, it is further classified into in-degree (incoming edges) and out-degree (outgoing edges).
- **Path:** A sequence of vertices where each adjacent pair is connected by an edge.
- **Cycle:** A path that starts and ends at the same vertex without repeating any vertices (except the starting/ending vertex).

Types of Graphs

1. **Undirected Graph:** Edges have no direction. Example: A friendship graph where connections are mutual.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1// Undirected Graph Representation
2class Graph {
3    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
4
5    public void addEdge(int v1, int v2) {
6        adjList.putIfAbsent(v1, new ArrayList<>());
7        adjList.putIfAbsent(v2, new ArrayList<>());
8        adjList.get(v1).add(v2);
9        adjList.get(v2).add(v1); // Undirected
10    }
11}
```

2. **Directed Graph (Digraph):** Edges have a direction. Example: A web page link structure.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1// Directed Graph Representation
2class DirectedGraph {
3    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
4
5    public void addEdge(int from, int to) {
6        adjList.putIfAbsent(from, new ArrayList<>());
7        adjList.get(from).add(to); // Directed
8    }
9}

```

3. **Weighted Graph:** Edges have weights (costs). Example: A road network where weights represent distances.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class WeightedGraph {
2    private final Map<Integer, List<Edge>> adjList = new
HashMap<>();
3
4    static class Edge {
5        int to;
6        int weight;
7
8        Edge(int to, int weight) {
9            this.to = to;
10           this.weight = weight;
11        }
12    }
13
14    public void addEdge(int from, int to, int weight) {
15        adjList.putIfAbsent(from, new ArrayList<>());
16        adjList.get(from).add(new Edge(to, weight));
17    }
18}

```

4. **Unweighted Graph:** Edges do not have weights. Example: A simple social network graph.

- **Example:** Similar to the undirected graph example above.

5. **Complete Graph:** Every pair of vertices is connected by an edge. Example: A tournament graph.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class CompleteGraph {
2    private final int vertices;
3
4    public CompleteGraph(int vertices) {
5        this.vertices = vertices;
6    }
7
8    public void printEdges() {
9        for (int i = 0; i < vertices; i++) {
10            for (int j = i + 1; j < vertices; j++) {
11                System.out.println("Edge: " + i + " - " + j);
12            }
13        }
14    }
15}

```

```

13     }
14 }
15}

```

6. **Bipartite Graph:** Vertices can be divided into two disjoint sets such that every edge connects a vertex in one set to a vertex in the other.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class BipartiteGraph {
2    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
3
4    public void addEdge(int u, int v) {
5        adjList.putIfAbsent(u, new ArrayList<>());
6        adjList.putIfAbsent(v, new ArrayList<>());
7        adjList.get(u).add(v);
8        adjList.get(v).add(u);
9    }
10}

```

7. **Directed Acyclic Graph (DAG):** A directed graph with no cycles. Example: Task scheduling.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class DAG {
2    private final Map<Integer, List<Integer>> adjList = new
HashMap<>();
3
4    public void addEdge(int from, int to) {
5        adjList.putIfAbsent(from, new ArrayList<>());
6        adjList.get(from).add(to); // Directed
7    }
8}

```

Conclusion

Graphs are versatile data structures used in various applications, from social networks to transportation systems. Understanding the terminology and types of graphs is crucial for implementing algorithms and solving problems effectively in Java DSA.

Graph Representation Using Adjacency Matrix and Adjacency List in Java

1. Adjacency Matrix Representation

An adjacency matrix is a 2D array where the rows and columns represent the vertices of the graph. If there is an edge between vertex `i` and vertex `j`, the matrix entry `matrix[i][j]` is set to `1` (or `true`), otherwise it is `0` (or `false`).

Java Code Example for Adjacency Matrix

```
java
VerifyOpen In EditorRunCopy code
1public class Graph {
2    private boolean[][] adjMatrix;
3    private int numVertices;
4
5    // Constructor to initialize the graph with a given number of vertices
6    public Graph(int numVertices) {
7        this.numVertices = numVertices;
8        adjMatrix = new boolean[numVertices][numVertices];
9    }
10
11    // Method to add an edge between two vertices
12    public void addEdge(int i, int j) {
13        adjMatrix[i][j] = true; // For directed graph
14        adjMatrix[j][i] = true; // Uncomment for undirected graph
15    }
16
17    // Method to remove an edge between two vertices
18    public void removeEdge(int i, int j) {
19        adjMatrix[i][j] = false;
20        adjMatrix[j][i] = false; // Uncomment for undirected graph
21    }
22
23    // Method to print the adjacency matrix
24    public void printGraph() {
25        for (int i = 0; i < numVertices; i++) {
26            for (int j = 0; j < numVertices; j++) {
27                System.out.print((adjMatrix[i][j] ? 1 : 0) + " ");
28            }
29            System.out.println();
30        }
31    }
32
33    public static void main(String[] args) {
34        Graph graph = new Graph(4);
35        graph.addEdge(0, 1);
36        graph.addEdge(1, 2);
37        graph.addEdge(2, 0);
38        graph.addEdge(2, 3);
39
40        System.out.println("Adjacency Matrix:");
41        graph.printGraph();
42    }
43}
```

2. Adjacency List Representation

An adjacency list is a collection of lists or arrays where each list corresponds to a vertex and contains a list of its adjacent vertices. This representation is more space-efficient for sparse graphs.

Java Code Example for Adjacency List

```
java
VerifyOpen In EditorRunCopy code
1import java.util.ArrayList;
2
3public class Graph {
4    private ArrayList<ArrayList<Integer>> adjList;
5    private int numVertices;
```



```

6
7 // Constructor to initialize the graph with a given number of vertices
8 public Graph(int numVertices) {
9     this.numVertices = numVertices;
10    adjList = new ArrayList<>(numVertices);
11    for (int i = 0; i < numVertices; i++) {
12        adjList.add(new ArrayList<>());
13    }
14 }
15
16 // Method to add an edge between two vertices
17 public void addEdge(int i, int j) {
18     adjList.get(i).add(j); // For directed graph
19     adjList.get(j).add(i); // Uncomment for undirected graph
20 }
21
22 // Method to print the adjacency list
23 public void printGraph() {
24     for (int i = 0; i < numVertices; i++) {
25         System.out.print(i + ": ");
26         for (int j : adjList.get(i)) {
27             System.out.print(j + " ");
28         }
29         System.out.println();
30     }
31 }
32
33 public static void main(String[] args) {
34     Graph graph = new Graph(4);
35     graph.addEdge(0, 1);
36     graph.addEdge(1, 2);
37     graph.addEdge(2, 0);
38     graph.addEdge(2, 3);
39
40     System.out.println("Adjacency List:");
41     graph.printGraph();
42 }
43}

```

Summary

- **Adjacency Matrix:** Suitable for dense graphs, allows $O(1)$ time complexity for edge existence checks, but uses $O(V^2)$ space.
- **Adjacency List:** More space-efficient for sparse graphs, allows easy iteration over neighbors, but edge existence checks take $O(V)$ time in the worst case.

Feel free to run the provided code examples to see how each representation works! If you have any further questions or need additional examples, let me know!

Graph Traversal Algorithms BFS and DFS in Java

1. Breadth-First Search (BFS)

BFS Overview:

- BFS is a traversal algorithm that explores all the neighbor nodes at the present depth prior to moving on to nodes at the next depth level.

- It uses a **queue** data structure to keep track of nodes to visit next.

BFS Algorithm Steps:

1. Start from the root node (or any arbitrary node in the case of a graph).
2. Enqueue the starting node and mark it as visited.
3. While the queue is not empty:
 - Dequeue a node from the queue.
 - Visit the node (process it).
 - Enqueue all its unvisited neighbors and mark them as visited.

Java Implementation of BFS:

```

1 java
2 VerifyOpen In EditorRunCopy code
3 import java.util.LinkedList;
4 import java.util.Queue;
5
6 class Graph {
7     private int V; // Number of vertices
8     private LinkedList<Integer>[] adj; // Adjacency list
9
10    Graph(int v) {
11        V = v;
12        adj = new LinkedList[V];
13        for (int i = 0; i < V; i++) {
14            adj[i] = new LinkedList<>();
15        }
16    }
17
18    void addEdge(int v, int w) {
19        adj[v].add(w); // Add w to v's list
20    }
21
22    void BFS(int start) {
23        boolean[] visited = new boolean[V]; // Track visited nodes
24        Queue<Integer> queue = new LinkedList<>(); // Queue for BFS
25
26        visited[start] = true; // Mark the starting node as visited
27        queue.add(start); // Enqueue the starting node
28
29        while (!queue.isEmpty()) {
30            int node = queue.poll(); // Dequeue a node
31            System.out.print(node + " "); // Process the node
32
33            // Enqueue all unvisited neighbors
34            for (int neighbor : adj[node]) {
35                if (!visited[neighbor]) {
36                    visited[neighbor] = true; // Mark as visited
37                    queue.add(neighbor); // Enqueue
38                }
39            }
40        }
41    }
42
43    public static void main(String[] args) {
44        Graph graph = new Graph(6);
45        graph.addEdge(0, 1);
46        graph.addEdge(0, 3);
47        graph.addEdge(0, 4);
48        graph.addEdge(4, 5);
49        graph.addEdge(3, 5);
50    }
51 }

```

```

48         graph.addEdge(1, 2);
49
50         System.out.println("Breadth First Traversal starting from vertex 0:");
51         graph.BFS(0);
52     }
53}

```

Output:

```

VerifyOpen In EditorRunCopy code
1Breadth First Traversal starting from vertex 0:
20 1 3 4 2 5

```

2. Depth-First Search (DFS)

DFS Overview:

- DFS is a traversal algorithm that explores as far as possible along each branch before backtracking.
- It can be implemented using recursion or a stack.

DFS Algorithm Steps:

1. Start from the root node (or any arbitrary node).
2. Mark the node as visited.
3. For each unvisited neighbor, recursively perform DFS.

Java Implementation of DFS (using recursion):

```

java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2
3class GraphDFS {
4    private int V; // Number of vertices
5    private LinkedList<Integer>[] adj; // Adjacency list
6
7    GraphDFS(int v) {
8        V = v;
9        adj = new LinkedList[V];
10        for (int i = 0; i < v; i++) {
11            adj[i] = new LinkedList<>();
12        }
13    }
14
15    void addEdge(int v, int w) {
16        adj[v].add(w); // Add w to v's list
17    }
18
19    void DFS(int node, boolean[] visited) {
20        visited[node] = true; // Mark the node as visited
21        System.out.print(node + " "); // Process the node
22
23        // Recur for all the vertices adjacent to this vertex
24        for (int neighbor : adj[node]) {
25            if (!visited[neighbor]) {
26                DFS(neighbor, visited); // Recursive call
27            }
28        }
29    }
30}

```

```

31     public static void main(String[] args) {
32         GraphDFS graph = new GraphDFS(6);
33         graph.addEdge(0, 1);
34         graph.addEdge(0, 3);
35         graph.addEdge(0, 4);
36         graph.addEdge(4, 5);
37         graph.addEdge(3, 5);
38         graph.addEdge(1, 2);
39
40         boolean[] visited = new boolean[6]; // Track visited nodes
41         System.out.println("Depth First Traversal starting from vertex 0:");
42         graph.DFS(0, visited);
43     }
44 }

```

Output:

[Verify](#)
[Open In Editor](#)
[Run](#)
[Copy code](#)
 1Depth First Traversal starting from vertex 0:
 20 1 2 3 5 4

Summary

- **BFS** is best for finding the shortest path in unweighted graphs and exploring nodes level by level.
- **DFS** is useful for exploring all possible paths and is often used in scenarios like maze solving, topological sorting, and cycle detection.

Feel free to ask if you have any questions or need further examples!